

Owners Forum Case Study Solution

Simple submission summary and the seven requested deliverable notes.

Summary: deterministic grounded assistant over 7 CSVs, optional post gate LLM rewrite layer, web app plus CLI, and 52 of 52 evaluation checks passing.

Run locally:

Web app	<code>python3 -m solution.web.server</code>
CLI	<code>python3 -m solution.assistant.cli</code>
Eval	<code>python3 -m solution.eval.run_eval</code>

1. Architecture note

Structured retrieval over the 7 CSVs: resolve entities, join rows, apply policy, then compose a cited answer.

The dataset is small and ID driven, so deterministic joins are simpler and more exact than vector retrieval here.

The LLM layer is optional and post gate only. It rewrites style, but does not retrieve or decide policy.

policy.py implements the provided policies.md, and data.py validates the CSV schema against data_dictionary.md.

2. Context and memory note

Context is session scoped and in process only: last resolved entities plus an in memory note store.

Pronoun carry over happens only when the new turn resolves nothing and the wording clearly refers back.

Clarify follow ups resume the original question when the user answers with a record ID or short disambiguator.

Notes that match no dataset entity stay unlinked rather than being attached by guesswork.

3. Retrieval strategy note

All CSVs are loaded into memory with primary key indexes and explicit join helpers, so retrieval paths stay readable and testable.

Entity resolution is tiered: record IDs first, then exact names, then limited partial name fallbacks.

Ambiguity is a first class result. The assistant returns options and asks instead of picking one.

Thematic questions use structured filters, negation checks, and evidence quality checks rather than semantic similarity.

4. Session notes note

Session notes live in a separate append only store and are never written back into the CSVs.

They use a separate citation namespace such as SN001 and always render as session only, unverified material.

Notes are surfaced alongside dataset facts, not merged into them.

When possible, a note links to resolved entity IDs. Otherwise it is stored unlinked.

5. Decision logic note

answer: enough grounded evidence, no unresolved ambiguity, citations attached to each claim.

clarify: multiple matching people, accounts, or events, so the assistant shows options and waits.

refuse: restricted contact detail or unsupported claim. It states why rather than going quiet.

escalate: sensitive or restricted material is in scope, so content is withheld and human review is flagged.

Stale, low reliability, unverified, or explicitly denied evidence is shown with caveats instead of being silently dropped.

6. Evaluation plan

The 15 provided prompts are the fixed regression set, and every run writes a transcript for review.

52 automated checks cover privacy, citation behavior, ambiguity, thematic filtering, and RSVP and contact edge cases.

The suite is aimed at judgment failures, not just crashes. Wrong but plausible answers are the main target.

With more time I would add per claim citation audits, prompt injection tests, and a small human rated usefulness pass.

7. Privacy and limitations note

Contact visibility is enforced in code before composition, so restricted details never depend on prompt compliance.

Restricted records are filtered before the optional LLM layer, so withheld content never enters model context.

The provided policies.md and data_dictionary.md are reflected directly in policy.py and data.py.

This is a trusted internal prototype, not a multi tenant system. Real deployment would need caller auth and role aware policy.

Intent detection is still keyword based, so unusual phrasing should eventually move to a classifier with confidence thresholds.

Source of record: solution/NOTES.md. This PDF is a plain text packaged version of the deliverable notes.